# Lecture 08: Python Basics (Chapter 6)

교재 Ch. 6.1

유준수 (Joon Soo Yoo, Ph.D.)

2026년 3월 19일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

# Chapter 6. Functions

## 6.1 함수 정의하기 (Defining Functions)

# 함수(Functions): 코드의 재사용

함수는 특정 작업을 수행하는 **이름이 붙은 코드 블록**입니다. 이미 우리는 `print()`, `input()`, `float()` 같은 함수들을 호출(Call)해 왔습니다.

- **반복 방지:** 같은 코드를 여러 번 쓸 필요 없이 이름만 부르면 됩니다.
- **모듈화(Modularity):** 복잡한 문제를 작은 단위의 기능으로 나눌 수 있습니다.
- **유지보수:** 함수 내부 코드 하나만 고치면, 그 함수를 쓰는 모든 곳이 업데이트됩니다.

# 함수 정의 문법 (Syntax)

def 키워드를 사용하여 새로운 함수를 만듭니다.

## 함수 정의와 호출

```
def welcome_class():   # 함수 정의 (Definition)
    """컴퓨터과학과 코딩 수업 환영 메시지 출력"""   # Docstring
    print("Welcome to 컴퓨터과학과 코딩수업 Class!")
welcome_class()   # 함수 호출 (Call)
```

## 함수 작성 규칙

| 항목 | 설명 |
|---|---|
| **snake_case** | 함수 이름은 소문자와 언더바(_)를 사용합니다. |
| **콜론(:)** | 정의문의 끝에는 반드시 콜론이 필요합니다. |
| **들여쓰기** | 함수의 몸체(Body)는 반드시 들여쓰기해야 합니다. |

# 함수가 주는 강력한 이점

로직이 수정될 때, 함수를 쓰지 않은 코드는 모든 부분을 찾아 고쳐야 하지만 함수는 단 한 곳만 수정하면 됩니다.

## 함수가 없는 코드

```
# 거리 계산 1
d1 = ((x2-x1)**2 + (y2-y1)**2)**0.5
# 거리 계산 2
d2 = ((x4-x3)**2 + (y4-y3)**2)**0.5
```

→ *계산 방식이 바뀌면 모든 줄을 수정해야 함*

## 함수를 쓴 코드 (DRY)

```
def calc_dist(x1, y1, x2, y2):
  return ((x2-x1)**2 + (y2-y1)**2)**0.5
d1 = calc_dist(x1, y1, x2, y2)
d2 = calc_dist(x3, y3, x4, y4)
```

→ *함수 안의 공식만 바꾸면 끝!*

* **DRY Principles:** *Don't Repeat Yourself!*

# Task 1: 영화관 매점 메뉴 (Cinema Concessions)

**Mission: 영화관의 메뉴를 출력하는 함수 `concessions()`를 작성하세요.**

1. 매개변수가 없는 함수 `concessions()`를 정의합니다.
2. 함수 내부에서 아래의 메뉴와 가격을 출력합니다.
3. 함수를 호출하여 결과를 확인하세요.

## 출력 예시

```
Food/Drink Options:
Popcorn: $8-10
Candy: $3-5
Soft drink: $5-7
```

# 6.2 Control Flow

함수의 순간이동과 복귀 원리

# 제어 흐름(Control Flow)의 기본 원칙

프로그램은 기본적으로 위에서 아래로 실행되지만, 함수를 만나면 흐름이 역동적으로 변합니다.

- **정의(Definition):** `def`로 시작하는 부분은 컴퓨터에게 "기술을 가르치는" 단계입니다. 호출 전까지는 실행되지 않습니다.
- **호출(Call):** 함수 이름을 부르는 순간, 제어권이 함수 내부로 이동합니다.
- **복귀(Return):** 함수의 마지막 줄을 실행하면, 다시 **호출되었던 지점 바로 다음 줄**로 돌아옵니다.

# 6.2.1 제어 흐름의 추적 (Tracing Control Flow)

프로그램의 실행 순서는 코드의 줄 번호(1, 2, 3...)와 다를 수 있습니다.

### 실행 코드

```
1: def greet():
2:   print("안녕!")
3:
4: print("시작")
5: greet()
6: print("끝")
```

### 실제 실행 순서 (Trace)

| 순서 | 줄 번호 | 상태 |
|---|---|---|
| **1** | 4 | "시작" 출력 |
| **2** | 5 | **함수 호출 (Jump!)** |
| **3** | 2 | "안녕!" 출력 (함수 내부) |
| **4** | 6 | **복귀 (Return)** 후 "끝" 출력 |

### 교수님의 포인트

1 2번 줄은 함수가 정의된 곳일 뿐, **호출되기 전까지는 실행되지 않고 그냥 지나칩니다.**

# 함수 안의 함수 (Nested Calls)

함수는 또 다른 함수를 호출할 수 있습니다. 모듈화를 통해 각 함수가 하나의 작은 작업만 담당하게 할 때 매우 유용합니다.

### 계층적 호출 구조

```
def accepted(): print("Accepted!")
def terms():
  ans = input("Accept? (Y/N): ")
  if ans == "Y": accepted()   # 또 다른 함수 호출
terms()   # 메인 호출
```

### 책갈피 원리

함수를 호출하는 것은 읽던 책에 **책갈피**를 끼워두고 다른 책(함수)을 보러 가는 것과 같습니다. 다른 책을 다 읽으면 책갈피가 있는 곳으로 정확히 돌아옵니다.

# Task 2: 약관 동의 시스템 업데이트

**Mission: `accepted()`와 `rejected()` 함수를 활용해 보세요.**

1. `accepted()`: "Thank you for accepting the terms." 출력
2. `rejected()`: "You have rejected the terms. Thank you." 출력
3. `terms()`: 사용자에게 Y/N을 묻고, 결과에 따라 위 두 함수 중 하나를 호출합니다.

# 6.3 Variable Scope

변수의 생존 범위: 전역(Global) vs 지역(Local)

# 변수 범위(Scope)란 무엇인가?

변수의 **범위(Scope)**는 프로그램 내에서 해당 변수에 접근할 수 있는 영역을 의미합니다.

- 변수가 어디서 선언되었느냐에 따라 그 '수명'이 결정됩니다.
- 파이썬은 변수를 찾을 때 **안쪽(Local)에서 바깥쪽(Global)** 순서로 찾아 나갑니다.

# 1. 전역 범위 (Global Scope)

함수 **외부**에서 생성된 변수로, 프로그램 전체 어디서든 접근이 가능합니다.

### 전역 변수의 예시

```
lab_name = "DAS Lab"   # 전역 변수
def print_lab():
    print(lab_name)   # 함수 내부에서도 접근 가능!
print_lab()
```

- 프로그램이 시작될 때 생성되어 종료될 때까지 유지됩니다.
- 공통 설정값이나 상수를 저장할 때 주로 사용합니다.

## 2. 지역 범위 (Local Scope)

함수 **내부**에서 생성된 변수로, 오직 해당 함수 안에서만 존재합니다.

### 지역 변수의 예시

```
def login():
    session_id = "A123"   # 지역 변수
    print(session_id)
login()
print(session_id)   # Error! 함수 밖에서는 보이지 않음
```

- 함수가 호출될 때 생성되고, 함수 실행이 끝나면 **즉시 소멸**합니다.

# 전역 변수 vs 지역 변수 비교

**Scope Comparison**

| 특징 | 전역 변수 (Global) | 지역 변수 (Local) |
|---|---|---|
| **선언 위치** | 함수 밖 | 함수 안 |
| **접근 범위** | 프로그램 전체 | 선언된 함수 내부만 |
| **생존 기간** | 프로그램 종료 시까지 | 함수 실행 종료 시까지 |
| **주요 용도** | 공유 데이터, 설정값 | 임시 계산, 함수 내 데이터 |

# 3. global 키워드: 함수 안에서 전역 변수 수정하기

기본적으로 함수 내부에서 전역 변수를 수정하려고 하면, 파이썬은 이를 동일한 이름의 **새로운 지역 변수**로 간주합니다. 진짜 전역 변수를 고치고 싶다면 `global` 키워드가 필요합니다.

## 전역 변수 수정하기

```
score = 0
def update_score():
    global score    # "바깥의 score를 쓰겠다"고 선언
    score += 10
update_score()
print(score)    # 결과: 10
```

# 4. 변수 이름 충돌 (Shadowing)

전역 변수와 지역 변수의 이름이 같으면, 함수 내부에서는 **지역 변수가 우선순위**를 가집니다. 이를 '가려짐(Shadowing)'이라고 합니다.

## 누구의 값일까요?

```
val = "Global"
def my_func():
    val = "Local"   # 전역 변수 val을 가림
    print(val)
my_func()   # 출력: Local
print(val)   # 출력: Global
```

# Task 3: 연구실 출입 권한 (Local Scope)

**Mission: 아래 조건에 따라 "가려짐(Shadowing)" 현상을 구현하세요.**

1. 전역 변수 `access_level = "Guest"`를 먼저 선언합니다.
2. 함수 `set_admin()`을 정의합니다.
3. 함수 내부에서 **지역 변수** `access_level`에 "Admin"을 할당하고 출력합니다.
4. 함수를 호출한 뒤, 함수 밖에서 다시 `access_level`을 출력해 보세요.

## 예상 출력 결과

```
함수 내부: Admin
함수 외부: Guest    # 전역 변수는 변하지 않아야 함!
```

## Any Questions?